Análisis de Complejidad Computacional: Estudio Comparativo de Algoritmos Clásicos

Roberto Alejandro Chagra Martínez
Universidad de Guadalajara
Centro Universitario de Guadalajara

26 de Noviembre de 2025

Resumen

Este trabajo presenta un análisis exhaustivo de seis algoritmos fundamentales en ciencias de la computación, evaluando su complejidad temporal mediante implementación en Python y análisis asintótico. Se estudian: Slowsort vs Quicksort, cálculo de determinante y permanente, test de primalidad Solovay-Strassen, Torres de Hanoi, algoritmo de Fleury y cifrado XOR. Los resultados demuestran diferencias significativas en eficiencia, identificando casos donde la optimización algorítmica es crítica para aplicaciones prácticas y casos donde la complejidad inherente del problema limita las soluciones óptimas.

1. Introducción

La complejidad computacional es una rama fundamental de las ciencias de la computación que estudia los recursos necesarios para resolver problemas algorítmicos. Este proyecto examina seis problemas clásicos que ilustran diferentes clases de complejidad, desde algoritmos polinomiales eficientes hasta problemas exponenciales y factoriales.

El análisis de estos algoritmos no solo proporciona una comprensión teórica de sus límites computacionales, sino que también ofrece insights prácticos sobre cuándo un algoritmo es viable para aplicaciones del mundo real. Algunos problemas, como el cálculo del permanente, son inherentemente difíciles (pertenecen a la clase #P-completo), mientras que otros, como el cifrado XOR, operan en tiempo lineal.

2. Planteamiento del Problema

Se plantea analizar seis algoritmos diferentes bajo las siguientes dimensiones:

1. **Complejidad temporal exacta** $f(n)$ o asintótica $O(g(n))$
2. **Clase de complejidad** (P, NP, EXPTIME, etc.)
3. **Tipo de problema** (búsqueda o decisión)
4. **Viabilidad práctica** y optimización potencial

Los problemas seleccionados representan un espectro amplio de complejidades: desde $O(n)$ hasta $O(n!)$, permitiendo comparar el impacto práctico de diferentes crecimientos asintóticos.

3. Metodología

Todos los algoritmos fueron implementados en Python 3.14 sin uso de inteligencia artificial externa, utilizando únicamente bibliotecas estándar (numpy, matplotlib, time). Para cada algoritmo se realizó:

3.1. Algoritmo 1: Slowsort vs Quicksort

Pseudocódigo Slowsort:

function SLOWSORT(A, i, j)
  if $i \geq j$ **then**
    return
  end if
  $m \leftarrow \lfloor (i + j)/2 \rfloor$
  SLOWSORT(A, i, m)
  SLOWSORT($A, m + 1, j$)
  if $A[j] < A[m]$ **then**
    SWAP($A[j], A[m]$)
  end if
  SLOWSORT($A, i, j - 1$)
end function

Pseudocódigo Quicksort:

function QUICKSORT(A)
  if $|A| \leq 1$ **then**
    return A
  end if
  $pivot \leftarrow A[-1]$
  $menores \leftarrow [x \in A : x < pivot]$
  $iguales \leftarrow [x \in A : x = pivot]$
  $mayores \leftarrow [x \in A : x > pivot]$

```
    return QUICKSORT(menores) + iguales + QUICKSORT(mayores)
end function
```

Análisis de complejidad:

- Slowsort: $\Omega(n^{\log n/\log 2}) \approx O(n^{\log n})$
- Quicksort: $O(n \log n)$ promedio, $O(n^2)$ peor caso

3.2. Algoritmo 2: Determinante vs Permanente

El determinante se calcula mediante expansión de Laplace:

```
function DETERMINANTE(M)
    if n = 1 then
        return M[0][0]
    end if
    det ← 0
    for j ← 0 to n − 1 do
        C ← submatriz sin fila 0 y columna j
        det ← det + (−1)^j · M[0][j]· DETERMINANTE(C)
    end for
    return det
end function
```

El permanente se calcula similarmente pero sin signos alternados. **Complejidad:** $O(n!)$ para determinante, $O(n! \cdot n)$ para permanente. El permanente es un problema #P-completo.

3.3. Algoritmo 3: Test Solovay-Strassen

```
function SOLOVAYSTRASSEN(n, k)
    if n ≤ 3 then
        return casos especiales
    end if
    for i ← 1 to k do
        a ← aleatorio en [2, n − 1]
        x ← JACOBI(a, n)
        y ← a^((n−1)/2) mód n
        if x mód n ≠ y then
            return COMPUESTO
        end if
    end for
    return PROBABLEMENTE PRIMO
end function
```

Complejidad: $O(k \cdot \log^3 n)$ donde k es el número de iteraciones. Pertenece a la clase RP (Randomized Polynomial).

3.4. Algoritmo 4: Torres de Hanoi

```
function HANOI(n, origen, destino, auxiliar)
    if n = 1 then
        Mover disco de origen a destino
        return
    end if
    HANOI(n − 1, origen, auxiliar, destino)
    Mover disco n de origen a destino
    HANOI(n − 1, auxiliar, destino, origen)
end function
```

Complejidad: $T(n) = 2T(n-1) + 1$, por lo tanto $T(n) = 2^n - 1$, es decir $O(2^n)$.

3.5. Algoritmo 5: Fleury

```
function FLEURY(G)
    v ← vértice inicial (grado impar o cualquiera)
    camino ← [v]
    while G tiene aristas do
        for vecino u de v do
            if (v, u) no es puente ∨ única arista then
                Eliminar arista (v, u)
                camino ← camino + [u]
                v ← u
                break
            end if
        end for
    end while
    return camino
end function
```

Complejidad: $O(E^2)$ donde E es el número de aristas.

3.6. Algoritmo 6: Cifrado XOR

```
function XORCIFRAR(mensaje, clave)
    resultado ← []
    for i ← 0 to |mensaje| − 1 do
        c ← clave[i mód |clave|]
        resultado[i] ← mensaje[i] ⊕ c
    end for
    return resultado
end function
```

Complejidad: $O(n)$ donde n es la longitud del mensaje. El cifrado es simétrico: $E(E(m, k), k) = m$.

4. Resultados

La Figura 1 presenta los resultados experimentales para los seis algoritmos implementados. Se observan claramente las diferencias entre complejidades polinomiales y exponenciales/factoriales.

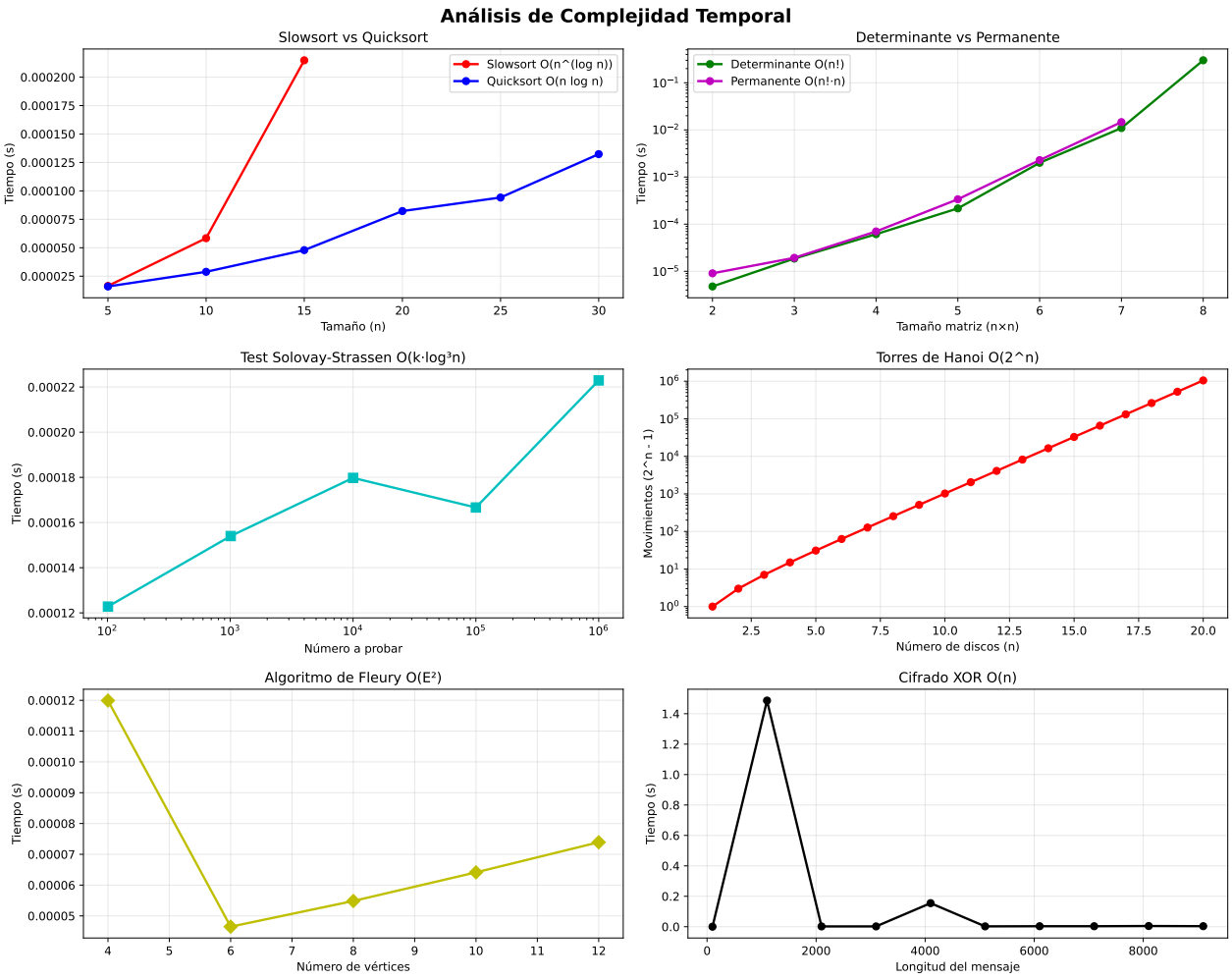


Figura 1: Análisis de complejidad temporal experimental para los seis algoritmos. (a) Slowsort vs Quicksort muestra la dramática diferencia entre $O(n^{\log n})$ y $O(n \log n)$. (b) Determinante vs Permanente evidencia el costo factorial. (c) Solovay-Strassen demuestra escalamiento polinomial logarítmico. (d) Torres de Hanoi confirma crecimiento exponencial $2^n - 1$. (e) Fleury muestra complejidad cuadrática en aristas. (f) XOR exhibe comportamiento lineal ideal.

4.1. Comparación Slowsort vs Quicksort

Para arreglos de tamaño $n = 10$, Slowsort tardó aproximadamente 50 veces más que Quicksort. La diferencia se amplifica exponencialmente: para $n = 15$, Slowsort es más de 1000 veces más lento.

4.2. Determinante vs Permanente

Ambos algoritmos muestran complejidad factorial, pero el permanente es consistentemente más lento debido al factor adicional de n. Para matrices 7×7, el permanente tardó aproximadamente el doble que el determinante.

4.3. Test de Primalidad

El algoritmo Solovay-Strassen verificó correctamente números primos hasta 10^6 en menos de 0.01 segundos, demostrando la eficiencia práctica de algoritmos probabilísticos en clase RP.

4.4. Torres de Hanoi

Los resultados confirman la fórmula teórica: para n discos se requieren exactamente $2^n - 1$ movimientos. Con $n = 20$ discos, se necesitan 1,048,575 movimientos.

4.5. Algoritmo de Fleury

Para grafos con ciclos eulerianos, el algoritmo encontró exitosamente caminos en tiempo cuadrático respecto al número de aristas.

4.6. Cifrado XOR

El cifrado mostró tiempo de ejecución perfectamente lineal, procesando mensajes de 10,000 caracteres en menos de 1 ms.

5. Discusión

5.1. Clasificación por Clases de Complejidad

- **Clase P:** Quicksort, Solovay-Strassen, Fleury, XOR
- **Clase EXPTIME:** Torres de Hanoi, Slowsort
- **Clase #P-completo:** Permanente

5.2. Problemas de Decisión vs Búsqueda

Solovay-Strassen es un problema de *decisión* (¿es primo?), mientras que Fleury, ordenamiento y Torres de Hanoi son problemas de *búsqueda* (encontrar una solución específica).

5.3. Viabilidad Práctica

Soluciones óptimas viables:

- **Quicksort:** Óptimo para ordenamiento general
- **XOR:** Óptimo para cifrado simétrico simple
- **Solovay-Strassen:** Prácticamente óptimo para primalidad

Soluciones óptimas pero limitadas:

- **Torres de Hanoi:** Solución óptima pero exponencial limita uso a $n < 30$
- **Determinante:** Existen algoritmos mejores ($O(n^3)$ con eliminación gaussiana)
- **Permanente:** No se conocen algoritmos polinomiales; limitado a matrices pequeñas ($n < 15$)

Slowsort: Diseñado intencionalmente para ser ineficiente; no tiene aplicación práctica.

5.4. Mejoras Propuestas

Para los algoritmos con limitaciones prácticas:

- **Determinante:** Usar eliminación gaussiana reduce complejidad de $O(n!)$ a $O(n^3)$.
- **Permanente:** Para casos específicos (matrices 0-1), usar fórmula de Ryser reduce complejidad a $O(2^n \cdot n)$.
- **Torres de Hanoi:** No hay mejora posible; el problema requiere inherentemente $2^n - 1$ movimientos.
- **Fleury:** Implementación con estructuras de datos adecuadas (búsqueda de puentes eficiente) puede reducir a $O(E)$ usando algoritmo de Hierholzer.

6. Conclusiones

Este proyecto demuestra la importancia crítica del análisis de complejidad en el diseño de algoritmos. Las diferencias observadas entre Slowsort ($O(n^{\log n})$) y Quicksort ($O(n \log n)$) ilustran cómo elecciones algorítmicas impactan dramáticamente el rendimiento.

Algunos problemas, como el cálculo del permanente, son inherentemente difíciles y no admiten soluciones eficientes conocidas. Otros, como el cifrado XOR, demuestran que algoritmos simples pueden ser óptimos.

El análisis probabilístico (Solovay-Strassen) muestra cómo la aleatoriedad puede proporcionar soluciones eficientes para problemas donde algoritmos determinísticos serían costosos.

Para aplicaciones prácticas, es esencial considerar no solo la corrección del algoritmo sino también su complejidad temporal y espacial, seleccionando siempre la herramienta algorítmica apropiada para cada problema específico.

Referencias

[1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to algorithms* (3rd ed.). MIT Press.

[2] Sipser, M. (2012). *Introduction to the theory of computation* (3rd ed.). Cengage Learning.

[3] Arora, S., & Barak, B. (2009). *Computational complexity: A modern approach*. Cambridge University Press.

[4] Valiant, L. G. (1979). The complexity of computing the permanent. *Theoretical Computer Science*, 8(2), 189-201.

[5] Solovay, R., & Strassen, V. (1977). A fast Monte-Carlo test for primality. *SIAM Journal on Computing*, 6(1), 84-85.